

FIAP Tech Challenge Fase 2

Pipeline batch para ingestao, transformacao e consulta de dados de mercado financeiro (B3) usando AWS.

1. Resumo Executivo

Este projeto implementa um pipeline de dados orientado a eventos para:

- coletar dados diarios de ativos via [yfinance](#);
- armazenar dados brutos em Parquet no Amazon S3 ([raw/](#));
- disparar processamento ETL no AWS Glue via evento de upload no S3;
- publicar camada refinada em Parquet particionada para consulta;
- disponibilizar analise SQL no Amazon Athena.

Resultado: arquitetura simples, auditavel e aderente aos requisitos do desafio, com separacao clara entre ingestao, orquestracao e transformacao.

2. Objetivo do Desafio

Atender ao enunciado do PDF [Tech_Challenge_Fase_2_Requisitos.pdf](#), cobrindo ponta a ponta:

- ingestao diaria;
- armazenamento particionado em S3;
- gatilho por evento;
- Lambda iniciando Glue Job;
- ETL com regras obrigatorias;
- publicacao refinada;
- catalogacao no Glue Data Catalog;
- consulta no Athena.

2.1 Escopo desta entrega

Arquitetura implementada e apresentada neste projeto:

- Script Bovespa (ingestao sob demanda);
- S3 Raw (bucket Bovespa);
- Lambda para trigger do Glue;
- Glue ETL Bovespa;
- S3 Refined;
- Athena para consulta.

3. Arquitetura da Solucao

```
flowchart LR
    A[Script Python\nIngestao yfinance] -->|Parquet| B[(S3 Raw\nraw/date=YYYY-MM-DD/ticker=...)]
    B -->|ObjectCreated| C[AWS Lambda\nStart Glue Job]
```

```
C --> D[AWS Glue ETL\nPySpark]
D -->|Parquet Particionado| E[(S3
Refined\nrefined/process_date=.../ticker=...)]
D --> F[Glue Data Catalog\nTabela refinada]
F --> G[Amazon Athena\nConsultas SQL]
```

3.1 Fluxo operacional

1. O script local executa a coleta dos tickers e grava Parquet no prefixo **raw/**.
2. O evento **ObjectCreated** do S3 aciona a Lambda.
3. A Lambda extrai a data da **key** e executa **glue:StartJobRun**.
4. O Glue processa os dados e grava a camada **refined/** com particoes.
5. O catalogo e atualizado (via crawler).
6. O Athena consulta a tabela refinada com filtro de particao.

4. Componentes do Projeto

4.1 Codigo versionado

- Script de ingestao: [app/ingest_bovespa.py](#)
- Dependencias: [pyproject.toml](#)

4.2 Servicos AWS utilizados

- Amazon S3 (camadas **raw/**, **refined/** e saida de consultas do Athena)
- AWS Lambda (orquestracao do Glue)
- AWS Glue (ETL e catalogacao)
- AWS Glue Data Catalog (metadados)
- Amazon Athena (consulta analitica)
- IAM + CloudWatch Logs (seguranca e observabilidade)

5. Estrutura de Dados no S3

5.1 Camada bruta (**raw**)

Padrao de escrita usado no projeto:

```
s3://bovespa-data/raw/date=YYYY-MM-DD/ticker=XXXX/dados.parquet
```

5.2 Camada refinada (**refined**)

Padrao de saida do ETL:

```
s3://bovespa-data/refined/process_date=YYYY-MM-DD/ticker=XXXX/part-*.parquet
```

Observacao: o enunciado cita `dt`. Nesta implementacao foi adotado `date/process_date` mantendo a mesma estrategia de particionamento por data + ticker.

6. Regras de Transformacao (Glue ETL)

O job ETL aplica as transformacoes exigidas no desafio:

- agregacao/sumarizacao por ticker (contagem, volume total, maximo, minimo);
- renomeacao de colunas:
 - `open` -> `opening_price`
 - `close` -> `closing_price`
- calculos temporais e derivacoes:
 - `price_range` = `high` - `low`
 - `daily_return` = `closing_price` - `opening_price`
 - media movel de 3 periodos (`moving_avg_3`) por ticker ordenado por data.

7. Aderencia aos Requisitos do PDF

Requisito	Como foi atendido
1. Ingestao diaria	Script Python com <code>yfinance</code> para coleta de series diarias
2. Raw no S3 particionado	Escrita Parquet no prefixo <code>raw/</code> com particao por data e ticker
3. Gatilho S3	Evento <code>ObjectCreated</code> no prefixo <code>raw/</code>
4. Lambda inicia Glue	Lambda faz apenas <code>StartJobRun</code> do Glue
5. ETL com 3 transformacoes	Agregacao, renomeacao e calculo temporal no Glue
6. Saida refinada	Parquet em <code>refined/</code> com particoes por data e ticker
7. Catalogo	Tabela registrada/atualizada no Glue Data Catalog
8. Consulta SQL	Athena consultando tabela refinada com filtro de particao

8. Execucao do Projeto

8.1 Pre-requisitos

- Python 3.13+
- Credenciais AWS configuradas (`aws configure`)
- Bucket S3 criado (ex.: `bovespa-data`)
- Permissoes IAM para S3, Glue, Lambda e Athena

8.2 Instalar dependencias

```
pip install boto3 pandas pyarrow yfinance
```

8.3 Executar ingestao local

```
python app/ingest_bovespa.py
```

8.4 Validar processamento na AWS

1. Confirmar arquivo no **raw/**.
2. Confirmar disparo da Lambda no CloudWatch.
3. Confirmar execucao do Glue Job.
4. Confirmar gravacao no **refined/**.
5. Confirmar tabela no Glue Data Catalog.
6. Executar query no Athena.

9. Consultas de Exemplo (Athena)

```
SELECT *  
FROM "AwsDataCatalog"."bovespa_db"."refined"  
WHERE process_date = '2026-03-23'  
LIMIT 10;
```

```
SELECT  
    ticker,  
    process_date,  
    date,  
    opening_price,  
    closing_price,  
    price_range,  
    daily_return,  
    moving_avg_3,  
    volume  
FROM "AwsDataCatalog"."bovespa_db"."refined"  
WHERE process_date = '2026-03-23'  
ORDER BY ticker, date DESC;
```

10. Seguranca e Governanca

Boas praticas adotadas:

- separacao de roles IAM por servico (Lambda e Glue);
- principio do menor privilegio (escopo por bucket/prefixo);
- logging centralizado no CloudWatch;
- particionamento para eficiencia de custo e performance no Athena.

11. Evidencias para Apresentacao

Checklist objetivo para demonstracao:

- ☐ arquivo Parquet em `raw/date=.../ticker=.../`
- ☐ evento S3 configurado para a Lambda
- ☐ log da Lambda com `StartJobRun`
- ☐ job Glue com transformacoes obrigatorias
- ☐ arquivo Parquet em `refined/process_date=.../ticker=.../`
- ☐ tabela no Glue Data Catalog
- ☐ query no Athena com resultado retornado

12. Roteiro de Apresentacao (5-8 minutos)

1. Contexto e objetivo do desafio (30-45s).
2. Arquitetura fim a fim (1-2 min, usar o diagrama e reforcar que o escopo e somente B3).
3. Demonstracao do fluxo: S3 -> Lambda -> Glue -> Catalog -> Athena (2-3 min).
4. Evidencia das transformacoes obrigatorias do ETL (1-2 min).
5. Fechamento com ganhos tecnicos: escalabilidade, rastreabilidade e custo (30-45s).

13. Melhorias Futuras

- orquestracao com EventBridge + Step Functions;
- CI/CD para versionamento de scripts Glue/Lambda;
- testes automatizados para contratos de schema e qualidade de dados;
- alarmes de falha em Lambda/Glue e monitoramento por metricas.